CHAPTER ONE

Object Oriented Analysis and Designing Basics

There are various concepts involved while developing a software, such as:

1. *Object-Oriented Analysis(OOA)*:It involves investigating a problem domain to identify the objects and their relationships that are relevant to the problem
2. *Object-oriented design (OOD)*: is the next step after OOA, which involves designing the software system based on the objects identified during OOA

Both combined is known as OOAD (Object-oriented analysis and design), which involves a number of techniques and practices, including object-oriented programming, design patterns, UML diagrams, and other use cases.

Here are some important aspects used in OOAD:

1. Object-Oriented Programming: involves modeling real-world objects as software objects, with properties and methods that represent the behavior of those objects
2. Design Patterns: Design patterns are reusable solutions to common problems in software design.
3. UML Diagrams: OOAD uses UML diagrams to represent the different components and interactions of a software system.
4. Use Cases: OOAD uses use cases to help developers understand the requirements of a system and to design software systems that meet those requirements.

   Advantages of OOAD.

   1. Reusability
   2. Scalability
   3. Maintainability
   4. Flexibility

   Disadvantages of OOAD.

   1. Complex to design
   2. Time-Complexity
   3. Difficult to make changes, once it has been designed
   4. Cost

In one line, it is tough to build something complex

Example of Object Oriented Design: Suppose you are designing a software system for a bank. You would start by performing object-oriented analysis to identify the objects and their relationships that are relevant to the problem. Some of the objects that you might identify include customers, accounts, and transactions. You would then use object-oriented design to create a software system based on these objects. For example, you might create a class called Customer that has properties such as name, address, and account_number. You might also create a class called Account that has properties such as balance, interest_rate, and account_type. You would then use these classes to create a software system that allows bank staff to manage customer accounts and transactions.

MOST USED TERMS IN OBJECT-ORIENTED DESIGNING:

1. *Objects*: is an instance of a class. It is a concrete entity that has its own state and behavior, based on the properties and methods defined in the class
2. *Class*: is a blueprint or template for creating objects. It defines the properties and methods that an object will have, but it is not an object itself.

3. *Messages*: Objects communicate by message passing. Messages consist of the integrity of the target object, the name of the requested operation, and any other action needed to perform the function
4. *Abstraction*: is the removal of the irrelevant and the amplification of the essentials.
5. *Encapsulation*: Encapsulation is also called an information hiding concept. Encapsulation not only bundles essential information of an object together but also restricts access to the data and methods from the outside world.
6. *Inheritance*: OOD allows similar classes to stack up in a hierarchical manner where the lower or sub-classes can import, implement, and re-use allowed variables and functions from their immediate super classes.
7. *Polymorphism*: OOD languages provide a mechanism where methods performing similar tasks but vary in arguments, can be assigned the same name. Depending upon how the service is invoked, the respective portion of the code gets executed.

If you are confused between Object and Class, because I was always confused in these concepts then you can remember this analogy:
Class provides the blueprint of the bicycle, but Object is the bicycle itself.

Doubt: How to design Objects?

We can Design an object in following phases:

1. *Identifying the Objects*
2. *Construction of design model for Object (Object Representation)*: *Static model* which describes the static structure using Class Diagram and Object Diagram. *Dynamic Model* which describes the dynamic structure of a system and show interaction between classes using Interaction diagrams and state-chart diagrams.
3. *Classification of Operations*: An operation specifies what is to be done and not how it should be done. So in this phase, the operations that would be performed on objects are defined...
4. *Algorithm Design*: The operations in the objects are defined using algorithms. An algorithm is a stepwise procedure that solves the problem laid down in an operation. Algorithms focus on how it is to be done. *Computation Complexity*, *flexibility of the Algorithm*, *Understandability* are the metrics for choosing the Optimal Algorithm
5. *Design of Relationships*: The strategy to implement the relationships needs to be chalked out during the object design phase.
6. *Implementation of control for external interactions*
7. *Package classes and associations into modules*: Hiding Internal Information from Outside view, Coherence of Elements, and Construction of Physical Modules

Design Optimizations methods:

1. Addition of Redundant Associations
2. Omission of Non-Usable Associations
3. Optimization of Algorithms
4. Saving and Storing of Derived Attributes

UML

Introduction

The UML is a graphical language for visualizing, specifying, constructing, and documenting the artifacts of a software-intensive system. The UML gives you a standard way to write a system's blueprints, covering conceptual things, such as business processes and system functions, as well as concrete things, such as classes written in a specific programming language, database schemas, and reusable software components.

Importance of modeling
A model is a simplification of reality. A model provides the blueprints of a system. A model may be structural, emphasizing the organization of the system, or it may be behavioral, emphasizing the dynamics of the system.

Aims of modeling
We build models so that we can better understand the system we are developing.

Through modeling, we achieve four aims.

1. Models help us to visualize a system as it is or as we want it to be.
2. Models permit us to specify the structure or behavior of a system.
3. Models give us a template that guides us in constructing a system.
4. Models document the decisions we have made.

We build models of complex systems because we cannot comprehend such a system in its entirety.

Principles of Modeling
There are four basic principles of model

1. The choice of what models to create has a profound influence on how a problem is attacked and how a solution is shaped.
2. Every model may be expressed at different levels of precision.
3. The best models are connected to reality.
4. No single model is sufficient. Every nontrivial system is best approached through a small set of nearly independent models.

Object Oriented Modeling
In software, there are several ways to approach a model. The two most common ways are

1. Algorithmic perspective
2. Object-oriented perspective

Algorithmic Perspective

- The traditional view of software development takes an algorithmic perspective.
- In this approach, the main building block of all software is the procedure or function.

- This view leads developers to focus on issues of control and the decomposition of larger algorithms into smaller ones.
- As requirements change and the system grows, systems built with an algorithmic focus turn out to be very hard to maintain.

Object-Oriented Perspective

- The contemporary view of software development takes an object-oriented perspective.
- In this approach, the main building block of all software systems is the object or class.
- A class is a description of a set of common objects.
- Every object has identity, state, and behavior.
- Object-oriented development provides the conceptual foundation for assembling systems out of components using technology such as Java Beans or COM+.

An Overview of UML

- The Unified Modeling Language is a standard language for writing software blueprints. The UML may be used to visualize, specify, construct, and document the artifacts of a software-intensive system.
- The UML is appropriate for modeling systems ranging from enterprise information systems to distributed Web-based applications and even to hard real time embedded systems. It is a very expressive language, addressing all the views needed to develop and then deploy such systems.

  The UML is a language for
  - Visualizing
  - Specifying
  - Constructing
  - Documenting

- Visualizing The UML is more than just a bunch of graphical symbols. Rather, behind each symbol in the UML notation is a well-defined semantics. In this manner, one developer can write a model in the UML, and another developer, or even another tool, can interpret that model unambiguously
- Specifying means building models that are precise, unambiguous, and complete.
- Constructing the UML is not a visual programming language, but its models can be directly connected to a variety of programming languages
- Documenting a healthy software organization produces all sorts of artifacts in addition to raw executable code. These artifacts include
  - Requirements
  - Architecture
  - Design
  - Source code
  - Project plans
  - Tests
  - Prototypes
  - Releases

Conceptual model of UML

To understand the UML, you need to form a conceptual model of the language, and this requires learning three major elements:

- Basic building blocks of the UML
- Rules
- Common Mechanisms in the UML

Basic building blocks of the UML: Vocabulary of the UML can be defined

1. Things
2. Relationships
3. Diagrams

Things in the UML

There are four kinds of things in the UML:

- Structural things
- Behavioral things
- Grouping things
- Annotational things

Structural things are the nouns of UML models. These are the mostly static parts of a model, representing elements that are either conceptual or physical. In all, there are seven kinds of structural things.

1. Classes
2. Interfaces
3. Collaborations
4. Use cases
5. Active classes
6. Components
7. Nodes

Class

- It is a description of a set of objects that share the same attributes, operations, relationships, and semantics.
- A class implements one or more interfaces.
- Graphically, a class is rendered as a rectangle, usually including its name, attributes, and operations.

.

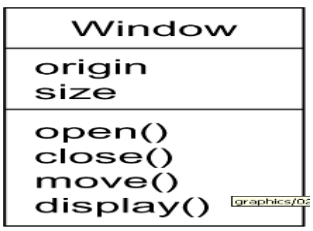
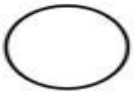


Fig: Class

Interface

- Interface is a collection of operations that specify a service of a class or component.
- An interface therefore describes the externally visible behavior of that element.
- An interface might represent the complete behavior of a class or component or only a part of that behavior.
- An interface is rendered as a circle together with its name. An interface rarely stands alone. Rather, it is typically attached to the class or component that realizes the interface.



IUnkonwn

Fig: Interface

Collaboration

- It defines an interaction and is a society of roles and other elements that work together to provide some cooperative behavior that's bigger than the sum of all the elements.
- Therefore, collaborations have structural, as well as behavioral, dimensions.
- A given class might participate in several collaborations.
- Graphically, a collaboration is rendered as an ellipse with dashed lines, usually including only its name.



Fig: Collaboration

Use case

- Use case is a description of set of sequence of actions that a system performs that yields an observable result of value to a particular actor
- Use case is used to structure the behavioral things in a model.
- A use case is realized by a collaboration. Graphically, a use case is rendered as an ellipse with solid lines, usually including only its name

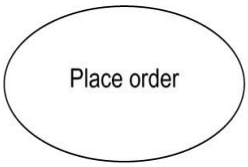


Fig: Use case

Active class

- It is just like a class except that its objects represent elements whose behavior is concurrent with other elements.
- Graphically, an active class is rendered just like a class, but with heavy lines, usually including its name, attributes, and operations.

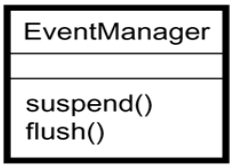


Fig: Active class

Component

- It is a physical and replaceable part of a system that conforms to and provides the realization of a set of interfaces.
- Graphically, a component is rendered as a rectangle with tabs.

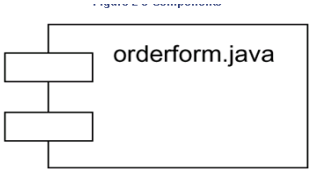


Fig: Component

Node

- It is a physical element that exists at run time and represents a computational resource, generally having at least some memory and, often, processing capability.
- Graphically, a node is rendered as a cube, usually including only its name.

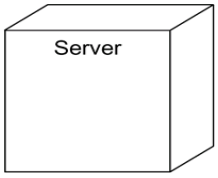


Fig: Node

Behavioral Things

- They are the dynamic parts of UML models. These are the verbs of a model, representing behavior over time and space.
- In all, there are two primary kinds of behavioral things. Interaction
  State machine

Interaction

- Interaction is a behavior that comprises a set of messages exchanged among a set of objects within a particular context to accomplish a specific purpose
- An interaction involves a number of other elements, including messages, action sequences and links
- Graphically a message is rendered as a directed line, almost always including the name of its operation



Fig: Message

State Machine

- State machine is a behavior that specifies the sequences of states an object or an interaction goes through during its lifetime in response to events, together with its responses to those events
- State machine involves a number of other elements, including states, transitions, events and activities
- Graphically, a state is rendered as a rounded rectangle, usually including its name and its sub states

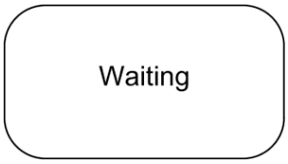


Fig: State machine

Grouping Things

1. These are the organizational parts of UML models. These are the boxes into which a model can be decomposed

2. There is one primary kind of grouping thing, namely, packages.

Package

- A package is a general-purpose mechanism for organizing elements into groups. Structural things, behavioral things, and even other grouping things may be placed in a package
- Graphically, a package is rendered as a tabbed folder, usually including only its name and, sometimes, its contents.



Fig: Package

Annotational things

- These are the explanatory parts of UML models. These are the comments you may apply to describe about any element in a model.
- There is one primary kind of Annotational thing, namely, note.

Note

- A note is simply a symbol for rendering constraints and comments attached to an element or a collection of elements.
- Graphically, a note is rendered as a rectangle with a dog-eared corner, together with a textual or graphical comment.

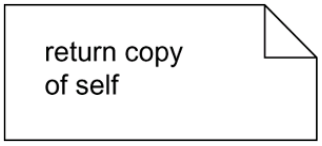


Fig: Note

Relationships in the UML:

There are four kinds of relationships in the UML:

1. Dependency
2. Association
3. Generalization
4. Realization

Dependency

- Dependency is a semantic relationship between two things in which a change to one thing may affect the semantics of the other thing
- Graphically a dependency is rendered as a dashed line, possibly directed, and occasionally including a label.

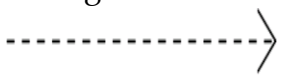


Fig: Dependency

Association

- Association is a structural relationship that describes a set of links, a link being a connection among objects.
- Graphically an association is rendered as a solid line, possibly directed, occasionally including a label, and often containing other adornments, such as multiplicity and role names.

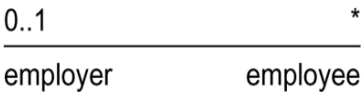


Fig: Association

Generalization

- Generalization is a special kind of association, representing a structural relationship between a whole and its parts.
- Graphically, a generalization relationship is rendered as a solid line with a hollow arrowhead pointing to the parent.

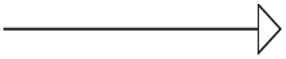


Fig: Generalization

Realization

- Realization is a semantic relationship between classifiers, wherein one classifier specifies a contract that another classifier guarantees to carry out.
- Graphically a realization relationship is rendered as a cross between a generalization and a dependency relationship.

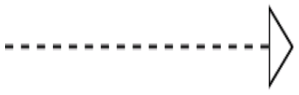


Fig: Realization